

Owner Full-Authority Configuration Architecture

Normalization, Idempotency, and the Retirement of `src/shared` Static Catalogs

YT Advertising Operations Control — Architecture Review

September 2026

Contents

1. Purpose and Scope	3
2. Why <code>src/shared</code> Is the Wrong Home for Business Data	4
2.1 Evidence from this codebase	4
2.2 The concrete cost of leaving it static	4
3. The Five Properties of a Fully Owner-Controlled Entity	6
4. Normalization Architecture	7
4.1 Principles	7
4.2 Concrete field-level normalization map	7
4.3 The ink-color regression as a worked example	7
5. Idempotency Architecture	9
5.1 Idempotent on retry (seed reruns, duplicate submissions)	9
5.2 Safe under concurrent edit (two people editing the same row)	9
5.3 Idempotency is not a substitute for an audit trail	9
6. Referential Integrity Guardrails	10
6.1 The pattern to apply everywhere	10
6.2 Reference audit — where this guard is needed	10
6.3 Reference checks must tolerate transitional and inactive state	10
7. The Governance Layer: Audit Trail and Segregation of Duties	11
7.1 Configuration change log	11
7.2 Elevated confirmation for the highest-risk entity	11
7.3 Segregation of duties in the UI itself	11
8. Roles and Permissions: The One Open Design Decision	12
8.1 Target schema	12
9. Sequenced Roadmap	13
Stage 1 — Close what's already load-bearing	13
Stage 2 — Services and materials to full normalization	13
Stage 3 — Roles and permissions (the governance-critical gap)	13
Stage 4 — Collapse remaining duplicate configuration surfaces	13

Stage 5 — Backfill, drift detection, audit trail 13

Stage 6 — Extensibility layer 13

10. Summary Checklist **14**

1. Purpose and Scope

This document consolidates the architecture direction for giving the **owner** full, no-code, no-deploy authority over every configurable business entity in the system — roles, permissions, services, materials, machines, routing rules, pricing, and production parameters — while retiring `src/shared` as a source of runtime business data.

It combines four threads of work already scoped for this codebase into one coherent plan:

1. The **database-first migration** (moving static catalogs into Convex tables).
2. The **normalization & idempotency** pattern (dropdown-driven master catalog work).
3. The **duplicated-configuration cleanup** (collapsing overlapping owner-settings sections onto single sources of truth).
4. The **governance guardrails** surfaced during the raw-material leakage review (audit trail, segregation of duties, referential integrity).

The guiding test used throughout: *would the owner ever want to add, rename, or remove an entry in this list without a developer touching code and redeploying?* Where the answer is yes, the data does not belong in a TypeScript file — it belongs in a table, with the five properties defined in Section 3.

2. Why `src/shared` Is the Wrong Home for Business Data

A file under `src/shared` tries to serve two audiences with fundamentally different lifecycles:

- The **frontend** wants synchronous, bundled, compile-time-typed data — no network round-trip, fast renders, safe autocomplete.
- The **backend** wants a single, owner-editable, runtime-authoritative source of truth that can change without a code deployment.

These two needs stay compatible only as long as the underlying list never needs to change without a developer. The moment the business needs a new service, a new material, a new role, or a renamed ink color, the two needs diverge — and the static file becomes debt. It is not a mistake to have written it; it is a mistake to leave it there once the business has outgrown it.

2.1 Evidence from this codebase

File	Lines	Import fan-out	Status
<code>src/shared/services.ts</code>	150	26 files	Highest-priority migration target
<code>src/shared/material-specifications.ts</code>	814	10 files	In progress (<code>materialCatalog</code>)
<code>src/lib/role-routing.ts</code>	627	5 files	Not yet migrated
<code>src/shared/role-permissions.ts</code>	193	2 files	Not yet migrated — this is the authorization system
<code>src/lib/access-policy.ts</code>	105	10 files	Depends on role-permissions migration
<code>src/shared/service-specifications.ts</code>	80	7 files	Not yet migrated
<code>src/lib/inventory-category-groups.ts</code>	301	1 file	Lowest priority
<code>src/shared/production-manifest.ts</code>	391	6 files	Partially superseded by <code>serviceRoutes</code>
<code>src/shared/roll-width.ts</code>	102	3 files	Cleanup once Phase 1 lands

Every one of these files has, at some point, required a “seed the database from this static array” migration step. That is the diagnostic signature of catalog data that was mislabeled as source code: **if a file has ever needed a script to copy its contents into a database, it was never really code.**

2.2 The concrete cost of leaving it static

Three real defects traced directly back to static-file duplication during this review, not hypothetical risk:

- **Three incompatible ink-color vocabularies** existed simultaneously — "CMYK"/"WHITE" (seeded directly into `machineInkConsumptionRules`), "Cyan"/"White" (derived by a name-substring migration into `materials.inkColor`), and free-text ("Eco-Solvent (CMYK)" embedded in `specificationValue`). A normalization fix shipped to address this and introduced a **new** regression: composite "CMYK" values collapse to a single "CYAN" channel at seed time, silently dropping Magenta/Yellow/Black consumption rules.
- **A documented risk control was never wired in.** `requireAdminPinForExceptions` and `maxDirectStockOutEtb` exist in `systemConfigs` but are never read by the mutation that performs exception stock-outs — the control exists in configuration but not in enforcement.
- **Waste margin, scrap limit, and ink consumption rate** are each configurable in two or three separate places (`production-section.tsx`, `scrap-allowance-section.tsx`, and the new `serviceRoutes` / `machineInkConsumptionRules` tables) that do not share a write path — changing one does not change the others.

Each of these traces back to the same root cause: business data was represented in more than one place because there was never a single authoritative table for it to live in.

3. The Five Properties of a Fully Owner-Controlled Entity

An entity is not “owner-controlled” until it has all five of the following. Missing any one of them means the entity is *owner-visible*, not *owner-authoritative*.

1. **A table.** The entity’s data lives in Convex — never in a `src/shared` array or object literal used as a runtime source.
2. **Owner-gated CRUD mutations.** Create, rename, and deactivate — never a hard delete (Section 6).
3. **Normalize-then-validate on write.** Trim and case-fold identity fields, reject near-duplicates, enforce field bounds and cross-field requirements (e.g. a `ROLL` material family requires `rollWidth`).
4. **Referential-integrity checks before destructive changes.** Before a rename or deactivation is allowed, every other table that references this entity (by id or, worse, by name string) must be checked, and the change blocked or flagged if something still points at the old value.
5. **A single UI home.** One slide-panel-driven surface per entity type, replacing every legacy free-text form that edited the same field elsewhere.

4. Normalization Architecture

4.1 Principles

- **Authoritative sources only.** Typed constants for anything genuinely fixed at compile time (design tokens, unit symbols) plus DB-backed catalogs for everything else. No invented strings at any call site.
- **Two enforcement layers.** UI dropdowns for the common path, and Convex mutation validators as the hard gate — so a pasted value, an API call, or a test write is normalized identically to a UI-driven edit.
- **Normalize, then validate.** Collapse whitespace, alias-map units and synonyms, canonicalize casing, *then* reject unknown enum values and unresolved references with an error that names the nearest valid match.
- **One normalizer, shared.** The same normalization module is imported by seed scripts, mutations, and (where relevant) migrations, so re-running a seed and performing a manual edit produce byte-identical canonical values.

4.2 Concrete field-level normalization map

Panel	Field	Normalizes against
Service	categoryKey / categoryNameEn	serviceCatalog categories (atomic pair)
Material	catalogFamily	ROLL \ RIGID_SHEET \ INK_SOLVENT \ HARDWARE
Material	baseUnit / purchaseUnit	Existing unit enums (nav-config.ts)
Material	category	Distinct values already in materialCatalog
Route	serviceId	serviceCatalog
Route	preferredMachineCode	machines.code
Route	preferredMaterialName	materialCatalog id/alias — not a free string match
Role	roleCode / workspaceId	Canonical role/workspace tables (Section 5)
Group	tone / iconName	Fixed design-token sets

4.3 The ink-color regression as a worked example

This is included because it is a live example of what “normalize without fixing the underlying representation” looks like, and the plan for every other entity should avoid repeating it:

- `normalizeInkColor("CMYK")` currently returns a single value ("CYAN"), because the composite-expansion helper (`normalizeCompositeInkColors`) exists but is never called from any production code path.
- The fix is not “add CMYK to the dropdown” — it is to decide, structurally, whether a machine’s ink rule should store **one row per channel** (recommended, since rates already differ — White ink is metered separately from CMYK in the seed data) or a

genuinely composite value with its own expansion logic used consistently at every read site.

- **General lesson for this architecture:** normalization work must always ask “does this field represent one atomic value, or a composite that different code paths are currently guessing at differently?” before building a dropdown over it.

5. Idempotency Architecture

Idempotency has two distinct guarantees that this codebase has so far only partially addressed, and they require different mechanisms:

5.1 Idempotent on retry (seed reruns, duplicate submissions)

- **Mechanism:** upsert-by-stable-key. Every entity gets a deterministic identity — either an owner-supplied slug or one derived from its name (lowercase, whitespace collapsed, non-alphanumerics stripped) — and every write path (seed, mutation, UI) looks the entity up by that key before deciding to insert or patch. `upsertMaterialCatalogItem`'s current insert-or-patch-by-id behavior is the correct template; it needs to be replicated for every other entity type, and the slug-derivation logic needs to live in one shared module so seed scripts and mutations can never derive two different keys for the same conceptual entity.
- **Verification:** re-running a seed script against an already-seeded database should report zero creates and only `updatedAt` changes where values genuinely differ — not a fresh flood of inserts.

5.2 Safe under concurrent edit (two people editing the same row)

This is the guarantee that is currently **missing everywhere**, including on the materials catalog panel already shipped. Upsert-by-key prevents duplicate rows on retry; it does nothing to stop a second save from silently overwriting a first one with no conflict signal.

- **Mechanism:** every entity's edit panel reads and holds the row's `updatedAt` when it opens. On save, the mutation receives that value as `expectedUpdatedAt`. If the stored `updatedAt` has since changed, the mutation rejects with a distinct, named error (e.g. `CONFIG_CONFLICT`) rather than patching — and the UI surfaces “this changed since you opened it, reload to see the latest” rather than a generic failure.
- **Why this matters more here than in most CRUD apps:** the explicit goal of this work is to make configuration editing frequent and low-friction for one person (the owner) — but also to open equivalent editing surfaces to admins/managers where appropriate. Frequent editing by more than one authorized party is exactly the condition under which silent last-write-wins causes real damage (a waste-margin correction quietly reverted, a role's permissions quietly reset).

5.3 Idempotency is not a substitute for an audit trail

An idempotent upsert guarantees the *shape* of the data is correct after repeated writes — it says nothing about *who* changed *what*, *when*, or *from what value*. Section 7 treats this as a separate, mandatory property.

6. Referential Integrity Guardrails

The single most consequential defect found in this review was not a missing field — it was a reference stored as a free-text string instead of a foreign key: `serviceRoutes.preferredMaterialName` matches materials by name, not by `materialCatalog.id`. Renaming a material through the new owner UI can silently break production routing for any service that still points at the old name, with no error at the time of the rename.

6.1 The pattern to apply everywhere

Before any entity’s **rename** or **deactivate** mutation is allowed to proceed:

1. Query every table known to reference this entity (by id where the schema already supports it, by normalized name-string match where it does not yet).
2. If active references exist, either:
 - **Block** the change and return an error listing every affected reference (the conservative default), or
 - **Cascade** the change if the reference has already been converted to a real foreign key (the preferred long-term state).
3. Never allow a hard delete of an entity that has ever been referenced — soft `active: false` only. History matters more than a clean table.

6.2 Reference audit — where this guard is needed

Referencing field	Currently	Target
<code>serviceRoutes.preferredMaterialName</code>	Free-text name match	FK to <code>materialCatalog.id</code> , guard already partially built
<code>machine.inkRequirements[].materialName</code> (legacy fallback path)	Case-insensitive name match	FK to <code>materialCatalog.id</code> , or retire the fallback path entirely
Any future <code>roleWorkspaceConfig.roleCode</code>	Free string	FK to a canonical <code>roles</code> table (Section 8)
Any future permission-matrix row	Free string role/permission pair	FK to canonical <code>roles</code> and a canonical <code>permissions</code> enum

6.3 Reference checks must tolerate transitional and inactive state

A reference check that hard-fails against *inactive* targets will break every seed rerun during a migration window. The check should distinguish “target does not exist” (hard error) from “target exists but is inactive” (warn, don’t block) — this is what allows the migration itself to proceed incrementally rather than requiring a single atomic cutover.

7. The Governance Layer: Audit Trail and Segregation of Duties

Granting full configuration authority without this section reproduces, at the configuration layer, exactly the accountability gap the earlier raw-material leakage review identified at the inventory layer: changes with real business consequence and no trace of who made them or why.

7.1 Configuration change log

Every owner-CRUD mutation in this architecture should write one entry to a new `configChangeLog` table:

```
configChangeLog: defineTable({
  entityType: v.string(),          // "material" | "service" | "role" | "route" | ...
  entityId: v.string(),
  action: v.union(v.literal("create"), v.literal("update"), v.literal("deactivate")),
  fieldChanges: v.optional(v.record(v.string(), v.object({
    from: v.any(),
    to: v.any(),
  }))),
  changedBy: v.string(),
  changedAt: v.number(),
})
```

This is inexpensive to add now, alongside each entity's migration, and expensive to retrofit later once every mutation site has to be revisited a second time.

7.2 Elevated confirmation for the highest-risk entity

The role → permission matrix is the one entity where a mistaken or malicious edit does not just misroute a job — it can silently grant access that should not exist. This entity should require a second confirmation step (mirroring the PIN-gate pattern already scoped for exception stock-outs) before a permission grant takes effect, in addition to the standard audit log entry.

7.3 Segregation of duties in the UI itself

Where practical, the person proposing a configuration change (e.g. a manager requesting a new material) and the person with final authority to activate it (the owner) should be distinguishable in the UI and in the audit log — even if, today, one person holds both roles. Building this distinction into the data model now costs little; retrofitting it after multiple people have owner-equivalent access costs a great deal more.

8. Roles and Permissions: The One Open Design Decision

Before migrating `ALL_ROLES` and `role-permissions.ts` into tables, one question needs an explicit answer, because it changes the shape of the schema and the UI:

Does “the owner can add a role with zero code” mean:

- **(a) Data-only roles** — a new row that reuses existing workspace types and existing permission primitives (e.g. “give this new role access to the operator workspace with `job.view` and `material.request` permissions”). This is fully achievable with the table-plus-CRUD pattern in this document.
- **(b) Structural roles** — a role that implies genuinely new UI, new machine-type awareness, or new business logic (e.g. a role tied to a workflow that does not yet exist). This is not a data problem, and no amount of configuration architecture makes it a no-code change.

Recommendation: build (a) as the real “full authority” feature, and make the distinction visible in the product itself — a role can be created freely when it maps to existing workspaces and permissions; a request that implies a new workspace or new logic should route to a “request a new capability” flow rather than silently failing or being misrepresented as possible through the form.

8.1 Target schema

```
roles: defineTable({
  code: v.string(),           // stable slug, e.g. "crystal_jet_operator"
  labelEn: v.string(),
  labelAm: v.string(),
  workspaceId: v.string(),    // FK-like reference to workspaceRoutes
  active: v.boolean(),
  createdAt: v.number(),
  updatedAt: v.number(),
})

rolePermissions: defineTable({
  roleCode: v.string(),       // FK -> roles.code
  permission: v.string(),     // FK -> a canonical permission enum
  active: v.boolean(),
})

.index("by_role", ["roleCode", "active"])
.index("by_permission", ["permission"])
```

9. Sequenced Roadmap

Ordered by dependency first, then by which entity currently causes the most real owner friction — not by which was easiest to discover during this review.

Stage 1 — Close what's already load-bearing

- Fix the ink-color composite-CMYK regression (Section 4.3) before it corrupts another seed run.
- Add `thickness` and a generic `attributes` field to `materialCatalog`.
- Add `expectedUpdatedAt` concurrency checks to the materials panel (already shipped elsewhere without this guard).

Stage 2 — Services and materials to full normalization

Shared normalizer module → mutation-side hard gates → convert every remaining free-text panel field to a dropdown or combobox, per the field map in Section 4.2.

Stage 3 — Roles and permissions (the governance-critical gap)

Migrate `ALL_ROLES` and `role-permissions.ts` into the schema in Section 8.1. Build the permission-matrix editor with the elevated-confirmation guard from Section 7.2. Resolve and productize the data-only-vs-structural-role decision from Section 8.

Stage 4 — Collapse remaining duplicate configuration surfaces

Fold per-material pricing (`override-section`, `valuation-section`, `etbValue`) into one documented precedence rule and one UI surface. Fold waste margin, scrap limit, and ink consumption rate down to `serviceRoutes` / `machineInkConsumptionRules` only, deleting the legacy admin-settings sections once every runtime reader is confirmed migrated.

Stage 5 — Backfill, drift detection, audit trail

Ship `configChangeLog` (Section 7.1) alongside every mutation touched in Stages 2–4. Ship a drift-detection report that flags existing rows whose values predate normalization, with a one-click, non-destructive migration path — never a silent or forced rewrite of historical data.

Stage 6 — Extensibility layer

Generalize the generic-attribute pattern (originally scoped for materials) to services and roles as well, so that the *next* new field this business needs is a data change made by the owner, not a schema migration made by a developer. This is the stage that actually fulfills the goal of this document — not just clearing the current backlog, but removing the conditions that produced it.

10. Summary Checklist

Use this as the acceptance criteria for declaring any entity “fully owner-authoritative”:

- ☐ Data lives in a Convex table, not a `src/shared` array or object literal.
- ☐ Create / rename / deactivate mutations exist and are owner-gated.
- ☐ Identity fields are trimmed, case-folded, and checked for near-duplicates before insert.
- ☐ Numeric and enum fields are validated, including cross-field requirements.
- ☐ Every other table that can reference this entity is checked before a destructive change, with inactive-but-existing targets tolerated.
- ☐ The mutation accepts `expectedUpdatedAt` and rejects stale writes with a named conflict error.
- ☐ The mutation writes an entry to `configChangeLog`.
- ☐ There is exactly one UI surface for editing this entity — every legacy free-text form for the same field has been deleted, not merely deprecated.
- ☐ Re-seeding produces zero duplicate rows and only genuine `updatedAt` changes.

An entity that satisfies every line above no longer needs a developer for the owner to add, rename, or remove an entry — which was the original, and only, requirement.